

Rules: Collaborate as you like, but everyone must write up their own solutions. Cite **all** references that you use (books, people, yourself) at the end of your writeup. All proofs should contain: (1) a one-line phrase with your proof strategy, (2) an informal paragraph outlining your idea, and (3) the gritty details. Be as precise and rigorous as you can!

Please print out the solutions to each problem on a *separate* piece of paper. Homework is due at *beginning* of class.

1 A Necessary Question

Say that a variable A in CFG G is **necessary** if it appears in every derivation of some string $w \in G$. Define the language

$$NECESSARY_{CFG} = \{ \langle G, A \rangle \mid A \text{ is a necessary variable in } G \}.$$

Prove that $NECESSARY_{CFG}$ is Turing-Recognizable.

Solution:

If there is a string $w \in G$ for which some variable A is necessary, then either A is the start symbol, or any derivation of w uses a rule which contains A on the right hand side.

Case 1: A is the start symbol

If the language is nonempty, then the start symbol is always necessary because it is present in the derivation of any string. However, if $L(G) = \emptyset$, then there is no string $w \in L(G)$ which requires A to be generated (because there are no strings $w \in G$) so the start symbol is not necessary.

Case 2: A is not the start symbol

For A to be necessary there must exist some string $w \in L(G)$ such that every derivation of w uses a rule where A is on the right hand side. Such a w is often called a witness to the fact that A is necessary.

Remove from G all production rules which contain A on the right hand side and call the resulting grammar G' . Every string which can only be generated using one of the rules we have removed from the grammar is in $L(G)$ but not in $L(G')$. Therefore, any witness to the necessity of A is contained in $L(G) \setminus L(G')$.

An algorithm for deciding $NECESSARY_{CFG}$:

```

if  $A$  is the start symbol
then
  if  $L(G) = \emptyset$  (this is decidable by Theorem 4.8)
  then accept
  else reject

else
  Create an enumerator  $E$  for  $G$ 
  while  $E$  enumerates some string  $x \in L(G)$ 
    if  $x \notin L(G')$ 
    then accept ( $x$  is a witness)
    else continue
  reject (there are no witnesses)

```

To reach the last line, the while loop must terminate, but this only occurs when $L(G)$ is finite and nonempty. So the machine may run forever. This need not worry us as we are only concerned in recognizing the language, not deciding it. Whenever A is necessary there will be a witness, and E is guaranteed to eventually produce that witness. So for all accepting cases the algorithm will terminate.

2 Enumerators

For this problem, we will define a variant of Turing machines, called *enumerators*. Intuitively, an enumerator is a Turing machine with an attached printer, which it can use to print strings.

Formally, an enumerator is a tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{print}}, q_{\text{halt}})$, where Q, Σ, Γ are finite sets and:

1. Q is the set of states
2. Γ is the work tape alphabet
3. Σ is the output tape alphabet
4. $\delta : Q \times \Gamma \times \{L, R\} \times \Sigma_\epsilon$ is the transition function
5. $q_0 \in Q$ is the start state
6. q_{print} is the print state
7. q_{halt} is the halt state

The computation of E is defined as an ordinary Turing machine, except for the following: it has two tapes, a work tape and a print tape, both initially blank. At each step, the machine may write a symbol from Σ on the print tape or may write nothing, depending on the transition function. Whenever the state q_{print} is reached, the print tape is set to blank and the head returns to the left-hand end. The machine halts when q_{halt} is reached.

Show that a language is decidable if and only if some enumerator enumerates the language in length-lexicographic order.

NOTE: Length-lexicographic order sorts first by length and then alphabetically.

Solution:

Suppose that a language L is decidable. Let D be a Turing machine that decides L and let E be an enumerator which enumerates Σ^* in length-lexicographic order. (How would you construct such an enumerator?) We define E_L , a length-lexicographic enumerator for L , as follows:

- Simulate E
- Every time E prints a new string x , E_L prints the string x if and only if $D(x)$ accepts.

Now suppose that L has a length-lexicographic enumerator E_L . We construct a TM $D(x)$ that decides L by simulating E_L . Whenever E_L outputs a new string y , $D(x)$

- Accepts, if $y = x$.
- Rejects, if $y > x$.
- Keeps Simulating E_L , otherwise.

Note that this will work as long as we can always find a $y > x$ which will happen if the language is infinite. If the language is finite, then it's trivially decidable.

3 Decidable, Turing-Recognizable, and Undecidable

- (a) In the *silly Post Correspondence Problem*, *SPCP*, the top string in each pair has the same length as the bottom string. Show that the *SPCP* is decidable.

Solution:

The *silly Post Correspondence Problem* is decidable. Let us be given an instance P of *SPCP*. By the language definition:

$$P = \left\{ \begin{bmatrix} t_1 \\ b_1 \end{bmatrix}, \begin{bmatrix} t_2 \\ b_2 \end{bmatrix}, \dots, \begin{bmatrix} t_k \\ b_k \end{bmatrix} \right\}$$

where $t_i, b_i \in \Sigma^*$ and $|t_i| = |b_i|$ for all i , where $1 \leq i \leq k$.

Claim: P has a match if and only if there exists a domino $\begin{bmatrix} t_i \\ b_i \end{bmatrix}$ such that $t_i = b_i$.

Proof: Assume P has a match. Then it must start with some domino. But the lengths of the top and bottom strings on that first domino are the same, so it must be the case that the top string is the same as the bottom string.

Now assume that there exists a domino with the same top string and bottom string. This domino forms a trivial match of *SPCP*.

Finally, we can trivially check if there exists a domino in the *SPCP* instance with the same top and bottom string. Thus we have proven that *SPCP* is decidable.

- (b) Let $J = \{w \mid \text{either } w = 0x \text{ for some } x \in A_{\text{TM}}, \text{ or } w = 1y \text{ for some } y \in \overline{A_{\text{TM}}}\}$. Show that neither J nor $\overline{J}$ is Turing-recognizable.

Solution:

Assume for the sake of contradiction that J is Turing-recognizable (i.e. semi-decidable). Then there exists a Turing machine S that, when given w as input, always accepts if w is of the specified form. Recall that $x \in A_{\text{TM}}$ has the form $(\langle M \rangle, u)$ where M is a Turing machine and $u \in L(M)$. Similarly, $x \in \overline{A_{\text{TM}}}$ also has the form $(\langle M \rangle, u)$ where $u \notin L(M)$ or else x does not have the form $(\langle M \rangle, u)$, which can also be checked for.

We can show that, assuming S , it's possible to completely decide A_{TM} using the following procedure.

- (a) Given an instance of A_{TM} , M and u , construct $x' = (\langle M \rangle, u)$, $w'_0 = 0x'$, and $w'_1 = 1x'$.
- (b) Run one copy of S (call it S_1) on w'_0 and a second copy of S (called S_2) on w'_1 in parallel.
If $x' \in A_{\text{TM}}$, then S_1 must accept w'_0 and say so within finite time. If $x' \in \overline{A_{\text{TM}}}$, then S_2 must accept w'_1 and say so within finite time. We know that x' must either be in A_{TM} or $\overline{A_{\text{TM}}}$ by definition, so one of S_1 or S_2 must return an answer in finite time. Depending on which machine accepts, we know whether x' is in A_{TM} or not.
- (c) If S_1 returns accept, then return **accept**; $x' \in A_{\text{TM}}$. If S_2 returns accept, then return **reject**; $x' \notin A_{\text{TM}}$.

We have decided A_{TM} , which is impossible, so J must not be semi-decidable.

Turning to $\overline{J}$, we first define the set more precisely:

$$\overline{J} = \{w \mid w = 0x \text{ and } x \notin A_{\text{TM}}, \text{ or } w = 1y \text{ and } y \in A_{\text{TM}}\}^1$$

The argument for proving that $\overline{J}$ is not semi-decidable is similar to the argument for J above. Assume TM T semi-decides $\overline{J}$ and construct x' , w'_0 , and w'_1 as before. If copy T_1 accepts w'_0 , then return **reject**; $x' \notin A_{\text{TM}}$. If copy T_2 accepts w'_1 , then return **accept**; $x' \in A_{\text{TM}}$.

- (c) Let $\Gamma = \{0, 1, \sqcup\}$ be the tape alphabet for all Turing machines in this problem. Define the **busy bandicoot** function $B : \mathbb{N} \rightarrow \mathbb{N}$ as follows. For each value of k , let $B(k)$ be the maximum number of steps a k -state Turing machine will run before halting. Define the language

$$\text{BUSYBANDICOOT} = \{\langle n, s \rangle \mid B(n) = s\}.$$

Show that BUSYBANDICOOT is undecidable. You may ignore cases where the machine might not halt.

Solution:

Assume for the sake of contradiction that B is decidable. Then there exists a Turing machine T that when given $\langle n, s \rangle$ as input, always accepts or rejects depending on if $B(n) = s$ or not. Assuming T , it is possible to decide A_{TM} .

- (a) Let us be given an arbitrary M with m states, and a string w . By assumption, T is decidable, therefore we can find the solution to $B(m)$ in its enumeration.

¹Or else $w = 0z$ or $w = 1z$, where $z \neq (\langle M \rangle, u)$, which can also be checked for.

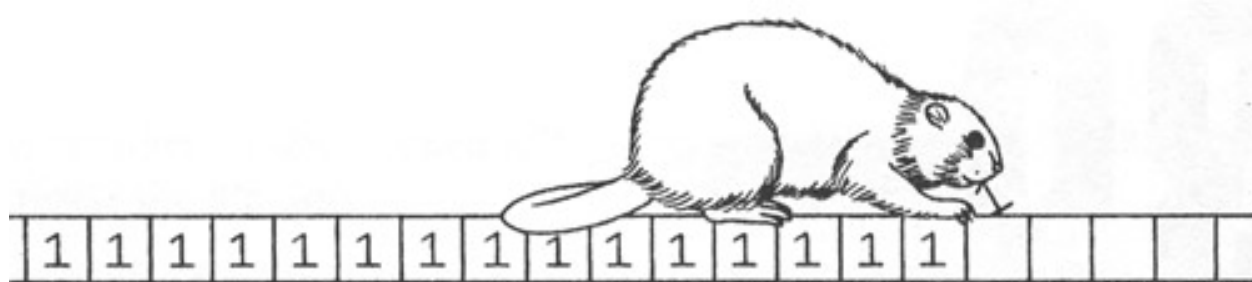
- (b) Run M on w for $B(m)$ steps. If M halts and accepts w , then accept. If M halts and rejects w , then reject. If M has not terminated after $B(m)$ steps, then we know the Turing machine has computed for the maximum number of steps before halting, so M on w must not terminate. Thus we reject.

Since this procedure always provides an answer for A_{TM} , which is not possible, we know that T must not exist. Therefore BUSYBANDICOOT is undecidable.

4 The Hard Question

A function $f : \Sigma^* \rightarrow \Sigma^*$ is a **computable function** if some Turing machine M , on every input w , halts with just $f(w)$ on its tape.

Let $\Gamma = \{0, 1, \sqcup\}$ be the tape alphabet for all Turing machines in this problem. Define the **busy beaver** function $BB : \mathbb{N} \rightarrow \mathbb{N}$ as follows. For each value of k , consider all k -state Turing machines that halt when started with a blank tape. Let $BB(k)$ be the maximum number of 1s that remain on the tape among all these machines. You may assume that the input to BB is in binary, and the output of the function is in unary. Your task is to prove that BB is not a computable function.



- (a) Show that for any natural number m there is a Turing machine without input using $\mathcal{O}(\log m)$ states that outputs m .

Solution:

We want to construct a machine that outputs m in $\mathcal{O}(\log m)$ states. We note that m written in binary has $\log m$ digits. Therefore we can construct a machine that, for a particular m , has $\log m$ states. State i reads a blank, prints out the i^{th} digit of m and transitions to state $i + 1$. Formally we can go by induction to show this construction holds for all m .

- (b) Assume $f : \mathbb{N} \rightarrow \mathbb{N}$ is a strictly increasing computable function. Show that for some sufficiently large x we must have $f(x) < BB(x)$.

Solution:

To prove for all strictly increasing computable functions f , there exists an x such that $f(x) < BB(x)$, we can first prove that BB is strictly increasing.

We want to show that for all n , $BB(n+1) > BB(n)$. $BB(n)$ is the maximum number of 1s that remain on the tape among all n -state Turing machines. Call the machine that achieves this m . Construct m' from m as from the following procedure.

- (a) Add a new state q_{n+1} .
- (b) Let all transitions to the accept or reject state transition to q_{n+1} .
- (c) If q_{n+1} reads a non-blank tape symbol, write 1, move right on the tape, and transition to itself.
- (d) If q_{n+1} reads a blank tape symbol, write 1 and transitions to q_{accept} .

Then m' has $n+1$ states and terminates with at least one additional 1 on the tape. The optimum $n+1$ -state machine leaves at least as many 1s on the tape as some $n+1$ -state machine, so it outputs a number at least as large as the output of m' . But the output of m' is greater than the output of the best n -state machine. Therefore, $BB(n+1)$, the output of the best $n+1$ -state machine, is strictly greater than $BB(n)$.

Since f is a strictly increasing computable function, there is a Turing machine M that computes it. Suppose M has c states. For large enough x , we can construct M_x , a machine that in $\log x$ states places the binary representation of x onto the tape and then runs f on the tape using c states. This machine takes no input and outputs $f(x)$ in $\log x + c$ states. Therefore $BB(\log x + c) \geq f(x)$ since $BB(k)$ returns the maximum number of 1s that can remain on the tape given k states.

Now since $\mathcal{O}(\log x)$ functions grow slower than $\mathcal{O}(x)$ functions, for large enough x , we have that $x > \log x + c$. By the lemma, for large enough x , $BB(x) > BB(\log x + c)$ so we conclude $f(x) < BB(x)$.

- (c) Conclude that BB is not computable.

Solution:

Assume for the sake of contradiction that BB is computable. $BB : \mathbb{N} \rightarrow \mathbb{N}$ and we have shown that it is strictly increasing. From part 2 we can conclude that there exists an x such that $BB(x) < BB(x)$. But since for all x we know that $BB(x) = BB(x)$ by the good old axioms of Peano arithmetic, we have reached a contradiction, so BB is not computable.

5

Put a References section at the end! Even if you worked alone, please cite yourself. For good academic practice, cite who you worked with **for each question**.